

Odpowiedzi do zadania

Jakość danych medycznych, model relacyjny, obrazy CT i architektura Data Lakehouse

Stos technologiczny open source

- PostgreSQL - relacyjna baza danych i warstwa metadanych.
- MinIO - storage obiektowy kompatybilny z S3 dla plików CT.
- Python - skrypty ETL, mapowanie CT i automatyzacja pipeline'u.
- Streamlit - prosty dashboard jakości danych.
- Docker Compose - uruchomienie środowiska w trzech kontenerach.
- OpenOffice / LibreOffice - konwersja MID.xlsx do CSV.
- SQL - transformacje, walidacje, raporty i zapytania kontrolne.

Architektura rozwiązania

Kontener / komponent	Rola w rozwiązaniu
Kontener 1: PostgreSQL	Przechowuje tabele relacyjne, dane leków, pacjentów testowych, wizyt, badań, metadane CT oraz problemy jakości danych.
Kontener 2: MinIO	Przechowuje pliki obrazów CT jako obiekty w bucketcie medical-lakehouse, np. bronze/ct/dicom/.
Kontener 3: App	Uruchamia Python, skrypty ETL, pipeline, upload CT do MinIO oraz dashboard Streamlit.

Przepływ danych

MID.xlsx -> OpenOffice/LibreOffice -> mid_raw.csv -> staging_mid_raw -> tabele relacyjne
CT z Kaggle -> data/raw/ct/ -> ct_patient_mapping.csv -> exams + ct_images -> MinIO + PostgreSQL

1. Zbiór tabelaryczny z Kaggle i opis struktury

W projekcie wykorzystano zbiór Medical Information Dataset z Kaggle w postaci pliku MID.xlsx. Plik został przekonwertowany do CSV i załadowany do tabeli staging_mid_raw. Główną encją w tym zbiorze jest lek / produkt medyczny. Jeden rekord opisuje jeden produkt leczniczy lub informację o leku.

Pole źródłowe	Znaczenie
Name	Nazwa leku lub produktu medycznego.
Link	Link źródłowy do strony z opisem leku.
Contains	Skład leku / substancje czynne.
ProductIntroduction	Ogólny opis produktu.
ProductUses	Zastosowanie leku.
ProductBenefits	Korzyści terapeutyczne.
SideEffect	Możliwe działania niepożądane.
HowToUse	Instrukcja stosowania.
HowWorks	Opis mechanizmu działania.
QuickTips	Krótkie wskazówki dla użytkownika.

SafetyAdvice	Zalecenia bezpieczeństwa.
Chemical_Class	Klasa chemiczna.
Habit_Forming	Informacja, czy lek może powodować uzależnienie.
Therapeutic_Class	Klasa terapeutyczna.
Action_Class	Klasa działania leku.

Potencjalne problemy jakości danych

- Braki danych w polach klasyfikacyjnych i opisowych, np. chemical_class, action_class, contains lub safety_advice.
- Duplikaty logiczne leków wynikające z różnej wielkości liter, nadmiarowych spacji lub wariantów nazw.
- Niespójne wartości słownikowe w polu Habit_Forming.
- Ślady HTML i scrapingu w polach opisowych.
- Niespójne klasy terapeutyczne, np. różne warianty nazw tej samej klasy.
- Problemy z formatem linków URL.
- Złożone wartości w polu Contains, gdzie jeden lek może zawierać kilka substancji czynnych.

Weryfikacja wzorców HTML wykazała, że problem dotyczył pól opisowych: product_introduction - 330 rekordów, quick_tips - 127 rekordów, safety_advice - 1 rekord. W polach kluczowych i słownikowych, takich jak name, link, contains, habit_forming, chemical_class, therapeutic_class i action_class, nie wykryto takich wzorców.

2. Proces zarządzania jakością danych medycznych

Dla zbioru zaproponowano proces jakości danych obejmujący etapy od profilowania do raportowania. Proces jest realizowany przez skrypty SQL, tabelę audytową data_quality_issues oraz dashboard Streamlit.

Etap	Opis realizacji
Profilowanie	Sprawdzenie liczby rekordów, kolumn, braków, wartości unikalnych i potencjalnych problemów w danych źródłowych.
Wykrywanie błędów	Reguły SQL wykrywają braki, duplikaty, niepoprawne formaty URL, wartości spoza słownika i ślady HTML.
Klasyfikacja problemów	Problemy są klasyfikowane jako CRITICAL, MAJOR albo MINOR, np. brak nazwy leku to CRITICAL, a ślady HTML w opisach to MINOR.
Korekty	Wykonywana jest standaryzacja nazw leków i substancji, normalizacja Habit_Forming, obsługa pustych wartości i podstawowe czyszczenie tekstu.
Walidacja	Weryfikowane są kluczowe pola, słowniki wartości, klucze obce i sztuczne mapowanie CT.
Raportowanie	Wyniki są dostępne w raportach SQL, dashboardzie Streamlit oraz tabeli data_quality_issues.

3. Trzy mierniki jakości danych

Dla tego zbioru najlepiej pasują trzy mierniki: kompletność, spójność oraz unikalność. Część mierników liczona jest na danych surowych w staging_mid_raw, a część po standaryzacji w tabelach docelowych.

Miernik	Sposób liczenia	Przykładowe zastosowanie
Kompletność	Liczba niepustych wartości w polu / liczba wszystkich rekordów * 100%.	Ocena braków w polach name, contains, chemical_class, action_class.

Spójność	Liczba rekordów zgodnych z regułą lub słownikiem / liczba wszystkich rekordów * 100%.	Weryfikacja, czy habit_forming ma wartości YES, NO lub UNKNOWN.
Unikalność	Liczba unikalnych wystandaryzowanych wartości / liczba wszystkich rekordów * 100%.	Ocena duplikatów nazw leków po medication_name_std.

Przykładowe zapytania

```
-- Kompletność
SELECT ROUND(100.0 * COUNT(chemical_class) / COUNT(*), 2) AS chemical_class_completeness_pct
FROM staging_mid_raw;

-- Spójność
SELECT ROUND(100.0 * SUM(CASE WHEN habit_forming IN ('YES','NO','UNKNOWN') THEN 1 ELSE 0 END) /
COUNT(*), 2) AS habit_forming_consistency_pct
FROM medications;

-- Unikalność
SELECT ROUND(100.0 * COUNT(DISTINCT medication_name_std) / COUNT(*), 2) AS medication_uniqueness_pct
FROM medications;
```

4. Relacyjna baza danych w PostgreSQL

Baza została zaprojektowana w PostgreSQL. Z jednej tabeli źródłowej MID.xlsx utworzono model relacyjny, ponieważ zadanie wymagało tabel pacjentów, wizyt, leków i badań. Dane leków pochodzą z MID.xlsx, natomiast pacjenci, wizyty i badania są testowe.

Tabela	Źródło / uzasadnienie
staging_mid_raw	Kopia 1:1 danych z CSV, odpowiadająca strukturze MID.xlsx.
medications	Główna tabela leków utworzona z pól Name, Link, Contains, Chemical_Class, Habit_Forming, Therapeutic_Class, Action_Class.
medication_descriptions	Długie opisy leków wydzielone z pól opisowych, np. ProductIntroduction, ProductUses, SideEffect, SafetyAdvice.
active_substances	Lista substancji czynnych wyprowadzona z pola Contains.
medication_substances	Tabela łącząca leki i substancje w relacji wiele-do-wielu.
patients	Pacjenci testowi utworzeni, ponieważ zadanie wymaga tabeli pacjentów, a MID.xlsx ich nie zawiera.
visits	Wizyty testowe wymagane przez model relacyjny zadania.
patient_medications	Relacja pacjent testowy - lek, potrzebna do zapytania lek -> CT.
exams	Tabela badań, w tym badań CT.
ct_images	Metadane obrazów CT i referencje do plików w storage obiekowym.
data_quality_issues	Tabela audytowa do zapisu wykrytych problemów jakości danych.

5. Ładowanie danych i preprocessing jakościowy

Dane zostały załadowane w ścieżce MID.xlsx -> CSV -> staging_mid_raw -> tabele relacyjne. Plik CSV jest importowany do PostgreSQL, a transformacje wykonuje SQL.

- Usunięcie / ograniczenie duplikatów: zastosowano pola techniczne medication_name_std i substance_name_std oraz unikalność logiczną po wartościach wystandaryzowanych.
- Standaryzacja wartości: usunięcie nadmiarowych spacji, ujednolicenie wielkości liter, normalizacja Habit_Forming do YES, NO, UNKNOWN.
- Obsługa braków danych: puste teksty są traktowane jako NULL albo UNKNOWN w zależności od charakteru pola.
- Walidacja kluczowych pól: sprawdzane są nazwy leków, linki, skład, wartości słownikowe, klucze obce oraz powiązania CT.
- Weryfikacja HTML: ślady HTML wykryto w product_introduction, quick_tips i safety_advice; problem jest rejestrowany jako UNCLEAN_TEXT.

Przykład standaryzacji:

Name -> medication_name_std

Contains -> substance_name_std

Habit_Forming -> YES / NO / UNKNOWN

Przykład wykrytego problemu tekstowego:

product_introduction: 330 rekordów z podejrzeniem HTML

quick_tips: 127 rekordów z podejrzeniem HTML

safety_advice: 1 rekord z podejrzeniem HTML

6. Obrazy CT i sztuczne mapowanie z pacjentami

Dodatkowy zbiór obrazów CT z Kaggle został rozpakowany do katalogu data/raw/ct/. Skrypt create_ct_mapping.py tworzy techniczne mapowanie obrazów CT do pacjentów testowych. Mapowanie jest sztuczne i służy wyłącznie do spełnienia wymagań zadania oraz przetestowania architektury.

Element	Opis
data/raw/ct/	Lokalny katalog z rozpakowanymi plikami CT, np. .dcm, .tif, .png, .jpg.
ct_patient_mapping.csv	Plik z technicznym powiązaniem obrazu CT z pacjentem testowym.
staging_ct_mapping	Tabela pośrednia do importu mapowania CT.
exams	Rekordy badań CT dla pacjentów testowych.
ct_images	Metadane obrazów CT oraz referencje storage_uri.
artificial_mapping_flag	Flaga TRUE wskazująca, że powiązanie CT jest sztuczne.

7. Przechowywanie CT w architekturze Data Lakehouse

Zaproponowano architekturę Data Lakehouse, w której pliki CT są przechowywane w storage obiektowym MinIO, a PostgreSQL przechowuje wyłącznie metadane i referencje do plików.

Warstwa	Zawartość
MinIO / storage obiektowy	Pliki CT, np. s3://medical-lakehouse/bronze/ct/dicom/ID_0000_AGE_0060_CONTRAST_1_CT.dcm.
PostgreSQL	Metadane CT: ct_image_id, exam_id, patient_id, file_name, storage_uri, modality, body_part, artificial_mapping_flag.
Warstwa bronze	Surowe pliki CT bez modyfikacji.
Warstwa silver / gold	Opcjonalne warstwy dla oczyszczonych lub gotowych do analizy danych.

W projekcie utworzono bucket medical-lakehouse w MinIO. Do demonstracji można załadować próbkę plików CT, 200 obiektów, a w bazie utrzymywać odpowiadające im storage_uri.

8. Raporty i dashboard

Rozwiązanie zawiera raporty SQL oraz prosty dashboard Streamlit. Raporty pokazują liczbę rekordów, kompletność danych, spójność, unikalność, problemy jakości oraz powiązania CT.

Raport / widok	Zakres informacji
record_counts	Liczba rekordów w tabelach staging_mid_raw, medications, patients, visits, exams, ct_images i data_quality_issues.
completeness_metrics	Kompletność najważniejszych pól, np. name, contains, chemical_class, action_class.
data_quality_issues_summary	Podsumowanie problemów jakości według severity i issue_type.
ct_mapping_summary	Liczba pacjentów z CT, liczba obrazów i liczba sztucznych mapowań.
dashboard Streamlit	Interaktywne widoki jakości danych, rekordów, braków i zapytania lek -> CT.

Dashboard: <http://localhost:8501>
MinIO: <http://localhost:9001>
PostgreSQL: localhost:5432

9. Zapytanie: CT pacjentów przyjmujących wskazany lek

Poniższe zapytanie wyszukuje badania CT pacjentów testowych przyjmujących lek pasujący do fragmentu nazwy XXX. Zapytanie łączy tabele patients, patient_medications, medications, exams i ct_images.

```
SELECT
  p.patient_id,
  p.pseudo_patient_code,
  m.medication_name,
  e.exam_id,
  e.exam_date,
  c.ct_image_id,
  c.storage_uri,
  c.artificial_mapping_flag
FROM patients p
JOIN patient_medications pm
  ON pm.patient_id = p.patient_id
JOIN medications m
  ON m.medication_id = pm.medication_id
JOIN exams e
  ON e.patient_id = p.patient_id
  AND e.exam_type = 'CT'
JOIN ct_images c
  ON c.exam_id = e.exam_id
WHERE m.medication_name_std LIKE LOWER('%XXX%')
LIMIT 20;
```

Wynik zapytania pokazuje techniczne powiązanie leku, pacjenta testowego, badania CT oraz referencji do pliku obrazu. Ponieważ pacjenci i mapowanie CT są sztuczne, wynik nie ma interpretacji klinicznej.